

Tracing Span Design: How Many Is Too Many

MAP CLARITY: INFORMATION HIERARCHY & USABLE DETAIL USABILITY CHALLENGE



Published on January 4, 2026



Webstack
Builders

Table of Contents

Span Fundamentals	3
Anatomy of a Span	4
Span Hierarchy and Context	6
Granularity Guidelines	8
What to Instrument	8
Span vs Event vs Attribute	10
Performance Overhead	13
Measuring Span Costs	13
Sampling Strategies	14
Trace Readability	16
Waterfall Anti-Patterns	16
Naming Conventions	18
Attribute Design	19
Selecting Attributes	20
Attribute Value Patterns	20
Practical Patterns	22
Request Lifecycle Instrumentation	22
Batch Processing Instrumentation	26
Conclusion	29



More spans provide more visibility. That's the intuition, anyway. But each span has costs: CPU overhead for creation, memory for attributes, network bandwidth for export, storage in your backend, and cognitive load when you're actually trying to read the trace. A request that creates 500 spans might have excellent granularity, but the trace waterfall becomes a solid block of color – no white space, no visible hierarchy. Debugging means scrolling through hundreds of spans looking for the slow one. Storage costs explode. The instrumentation itself becomes a performance concern.

Span design requires judgment, not just enthusiasm for visibility. The goal is *enough* spans to debug problems, not so many that you create new ones.

I learned this the hard way. A team I worked with instrumented a new service with spans for every function call, database query, cache lookup, and external request. Thorough, right? A typical request generated 200+ spans. The trace backend showed a waterfall of solid color – no gaps, no obvious structure. Finding the slow operation meant scrolling through hundreds of spans, mentally filtering out the noise. Storage costs tripled in a month.

WARNING

The span count that's "right" depends on your debugging needs. A payment service might need fine-grained spans to audit every step. A high-throughput cache might need minimal spans to avoid overhead. There's no universal number – but there are universal principles.

They refactored. Instrumented service boundaries, significant I/O operations, and error paths. Span count dropped to 15-20 per request. Traces became readable. The critical path was obvious at a glance. Storage costs dropped 90%. Debugging time went from minutes to seconds.

The lesson: granularity without readability is noise, not observability.

Span Fundamentals

Before diving into granularity decisions, it helps to understand what you're actually creating when you start a span. Every span carries overhead, and that overhead scales with how much data you attach to it.



Anatomy of a Span

A span has six core components:

1

Identity

Ties the span to its trace. The `trace_id` (16 bytes) is shared across every span in the trace – it's what lets your backend stitch together spans from different services. The `span_id` (8 bytes) uniquely identifies this span. The `parent_span_id` links to the parent span, creating the hierarchy that becomes your waterfall visualization.

2

Naming

Tells you what operation this span represents. The name should describe the operation (`HTTP GET /api/orders/{orderId}`), and the kind indicates the span's role in the request flow: `SERVER` for handling incoming requests, `CLIENT` for outgoing calls, `PRODUCER` and `CONSUMER` for async messaging, `INTERNAL` for operations that don't cross network boundaries.

3

Timing

Captures when the span started and ended, with nanosecond precision. Duration is simply end minus start – this is what your trace backend uses to identify slow operations.

4

Status

Records the outcome: `UNSET` (no status set), `OK` (operation succeeded), or `ERROR` (operation failed). For errors, you can include a message describing what went wrong.

5

Attributes

Key-value pairs that add context. Some follow OpenTelemetry semantic conventions (`http.method`, `db.system`), others are custom to your domain (`order.id`, `customer.tier`). Attributes are where most of your per-span overhead comes from – each attribute requires memory allocation and serialization.

6

Events

Timestamped logs within the span's lifetime. Use them for milestones: `cache.miss`, `retry.attempt`, `validation.failed`. Events are lighter than child spans but still have overhead.



7

Links

Connect spans across traces – useful when one request triggers another asynchronously. For example, when processing a message from a queue, you can link back to the span that originally enqueued it:

span-links.ts

```
1  import { trace, SpanKind, context } from '@opentelemetry/api';
2
3  const tracer = trace.getTracer('message-processor');
4
5  async function processMessage(message: Message) {
6    // Extract the trace context from the message that was enqueued earlier
7    const producerContext = propagation.extract(context.active(), message.headers);
8    const producerSpanContext = trace.getSpanContext(producerContext);
9
10   return tracer.startActiveSpan(
11     'message.process',
12     {
13       kind: SpanKind.CONSUMER,
14       links: producerSpanContext ? [{ context: producerSpanContext }] : [],
15     },
16     async (span) => {
17       span.setAttribute('message.id', message.id);
18       await handleMessage(message);
19       span.end();
20     }
21   );
22 }
```

Linking a consumer span to its producer.

Each of these components has a cost. Memory for the span object itself runs 200-500 bytes depending on attribute count. CPU cycles go to attribute serialization. Network bandwidth carries the span to your collector. Storage in your backend keeps it queryable. These costs are small individually, but they multiply by span count.



Span Hierarchy and Context

Spans form a tree. The root span represents the entry point – usually an incoming HTTP request. Child spans nest inside the root, representing operations that happen during request processing. This hierarchy is what creates the waterfall visualization in your trace UI.

The key mechanism is **context propagation**. When you call `startActiveSpan`, OpenTelemetry automatically makes that span the “current” span. Any child spans you create inherit the parent from this context. When you make an outgoing HTTP call, the trace context gets injected into headers so the downstream service can continue the trace.

Here’s what this looks like in practice – a request handler that checks cache, queries the database if needed, and enriches the result:

span-hierarchy.ts

```
1 // OpenTelemetry span hierarchy example
2 import { trace, SpanKind, SpanStatusCode } from '@opentelemetry/api';
3
4 const tracer = trace.getTracer('order-service');
5
6 async function handleGetOrder(req: Request, res: Response) {
7   return tracer.startActiveSpan(
8     'HTTP GET /orders/:id',
9     { kind: SpanKind.SERVER },
10    async (rootSpan) => {
11      try {
12        rootSpan.setAttribute('order.id', req.params.id);
13
14        // Child span created automatically under rootSpan
15        const cachedOrder = await checkCache(req.params.id);
16        if (cachedOrder) {
17          rootSpan.setAttribute('cache.hit', true);
18          return res.json(cachedOrder);
19        }
20
21        rootSpan.setAttribute('cache.hit', false);
22        const order = await fetchOrderFromDb(req.params.id);
23
24        rootSpan.setStatus({ code: SpanStatusCode.OK });
```



```

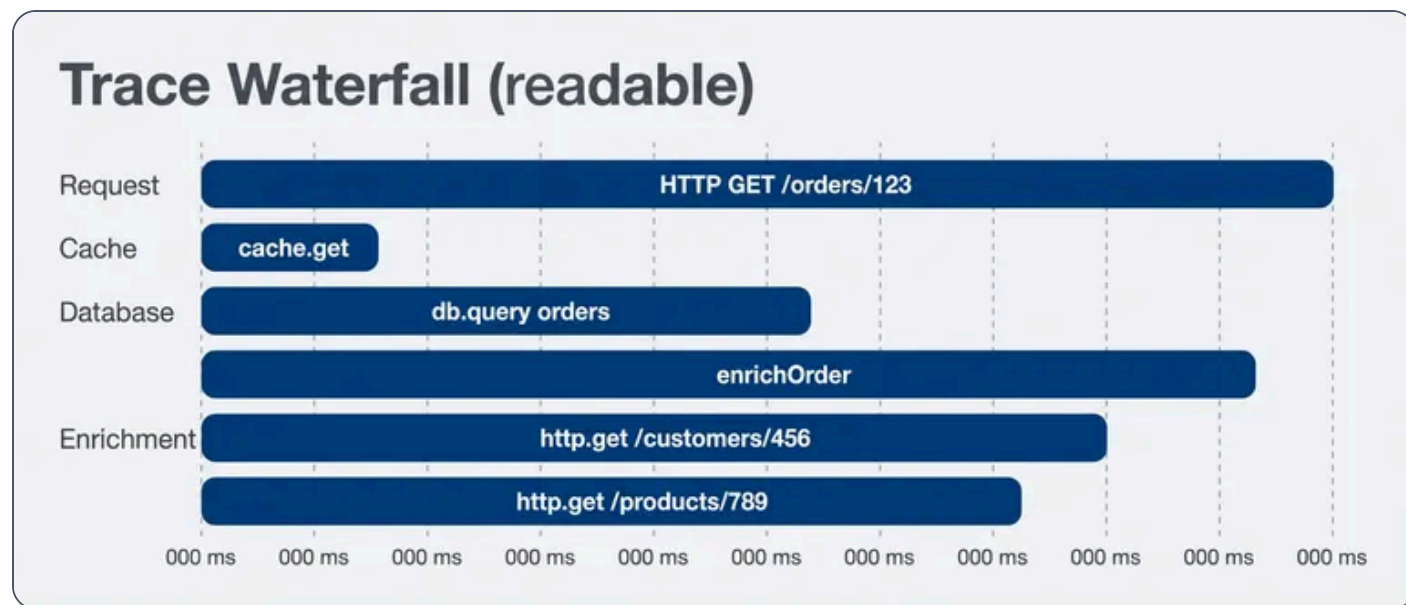
25         return res.json(order);
26     } catch (error) {
27         rootSpan.setStatus({ code: SpanStatusCode.ERROR, message: error.message });
28         rootSpan.recordException(error);
29         throw error;
30     } finally {
31         rootSpan.end();
32     }
33 }
34 );
35 }
36
37 async function checkCache(orderId: string): Promise<Order | null> {
38     return tracer.startActiveSpan('cache.get', { kind: SpanKind.INTERNAL }, async (span)
39 => {
40         try {
41             span.setAttribute('cache.key', `order:${orderId}`);
42             const result = await redis.get(`order:${orderId}`);
43             span.setAttribute('cache.hit', result !== null);
44             return result ? JSON.parse(result) : null;
45         } finally {
46             span.end();
47         }
48     });
49 }
50
51 async function fetchOrderFromDb(orderId: string): Promise<Order> {
52     return tracer.startActiveSpan('db.query', { kind: SpanKind.CLIENT }, async (span) => {
53         try {
54             span.setAttribute('db.system', 'postgresql');
55             span.setAttribute('db.operation', 'SELECT');
56             const result = await db.query('SELECT * FROM orders WHERE id = $1', [orderId]);
57             span.setAttribute('db.rows_affected', result.rowCount);
58             return result.rows[0];
59         } finally {
60             span.end();
61         }
62     });
63 }

```

Span hierarchy with parent-child relationships.



This produces a readable waterfall — you can see at a glance that the request spent time in cache lookup, then database, then enrichment. The gaps between spans show where time went to uninstrumented code (JSON parsing, business logic).



Readable trace waterfall with clear hierarchy.

INFO

Span hierarchy creates the waterfall visualization. Parent-child relationships show which operations are nested within others. The trace's critical path becomes visible when you can see which child spans are sequential vs parallel.

Granularity Guidelines

What to Instrument

The question “should this operation have a span?” comes up constantly. Here’s how I think about it.



Always instrument service boundaries.

Incoming HTTP / gRPC requests, outgoing HTTP / gRPC calls, message queue publish / consume – these are the spans that stitch your distributed trace together. Without them, your trace stops at service boundaries and you lose visibility into cross-service latency.

**Always instrument I/O operations.**

Database queries, cache reads / writes, file system operations. I/O is where latency lives. A trace that doesn't show database time is a trace that can't explain why a request was slow.

**Always instrument external dependencies.**

Third-party APIs, payment processors, cloud service SDKs. External services are the leading cause of incidents. When Stripe is slow or an S3 call times out, you want that visible in your traces.

**Instrument significant business operations.**

Order processing, payment validation, user authentication – operations that matter to your business and that you might need to debug. The keyword is "significant." Not every function, just the ones you'd want to see in a trace when something goes wrong.



Some operations need judgment:

Loops with external calls

Wrap the loop, not each iteration. A batch that processes 100 items should create one span for the batch, not 100 spans.

**Retry logic**

One span per attempt can be useful for debugging retry storms, but link them together so you can see the full retry sequence.

**Parallel operations**

A span for the fan-out operation, with child spans for each parallel branch.



Avoid instrumenting pure computation.

JSON parsing, field validation, data transformation – these are CPU-bound operations that rarely need their own spans. If you need timing, add it as an attribute on the parent span.



Avoid instrumenting every function call.

This is the trap that creates unreadable traces. Instrument boundaries and I/O, not internal implementation details.



Operation	Span?	Alternative	Why
HTTP request handler	Yes	---	Service boundary
Database query	Yes	---	I/O latency
JSON parsing	No	Attribute	Pure computation
Each loop iteration	No	Wrap loop	Overhead
External API call	Yes	---	Dependency visibility
Field validation	No	Attribute	Too granular
Error handling	No	Event	Within parent span

Span decision matrix.

Span vs Event vs Attribute

Once you've decided something deserves visibility in a trace, you have three options: span, event, or attribute. Each has different costs and use cases.



➤ Spans

Represent operations with duration. They have start and end times, can have child spans, and appear as bars in your waterfall. Use them for I/O operations and significant boundaries. Cost: highest (memory allocation, context propagation, serialization).

➤ Events

Timestamped points within a span's lifetime. They don't have duration – they mark moments. Use them for milestones: "validation started," "cache miss," "retry attempted." Cost: moderate (stored with parent span, but no separate context).

➤ Attributes

Key-value metadata attached to a span. They describe the operation: IDs, counts, flags, outcomes. Use them for context that helps you understand the span. Cost: lowest (just map entries).

Here's the pattern in practice:

span-event-attribute.ts

```

1  // Span vs event vs attribute in practice
2  import { trace, SpanKind, SpanStatusCode } from '@opentelemetry/api';
3
4  const tracer = trace.getTracer('order-service');
5
6  async function processOrder(order: Order) {
7    return tracer.startActiveSpan('processOrder', async (span) => {
8      // ATTRIBUTES: metadata about the operation
9      span.setAttribute('order.id', order.id);
10     span.setAttribute('order.item_count', order.items.length);
11
12     // EVENT: milestone within the span
13     span.addEvent('validation.started');
14     const validationResult = validateOrder(order);
15     span.addEvent('validation.completed', {
16       'validation.passed': validationResult.valid,
17     });
18
19     if (!validationResult.valid) {
20       span.setAttribute('validation.error', validationResult.error);
21       span.setStatus({ code: SpanStatusCode.ERROR });
22       span.end();
23       return;
24     }
25

```



```
26 // CHILD SPAN: I/O operation deserves its own span
27 await tracer.startActiveSpan('db.insertOrder', async (dbSpan) => {
28   dbSpan.setAttribute('db.system', 'postgresql');
29   await db.insert(order);
30   dbSpan.end();
31 });
32
33 span.addEvent('order.persisted');
34
35 // CHILD SPAN: external service call
36 await tracer.startActiveSpan('payment.charge', { kind: SpanKind.CLIENT }, async
(paymentSpan) => {
37   paymentSpan.setAttribute('payment.amount', order.totalAmount);
38   const result = await paymentService.charge(order);
39   paymentSpan.setAttribute('payment.transaction_id', result.transactionId);
40   paymentSpan.end();
41 });
42
43 span.setStatus({ code: SpanStatusCode.OK });
44 span.end();
45 });
46 }
47
48 // validateOrder() is pure computation - no span needed
49 function validateOrder(order: Order): ValidationResult {
50   const valid = order.items.length > 0 && order.totalAmount > 0;
51   return { valid, error: valid ? undefined : 'Invalid order' };
52 }
```

Using spans for I/O, events for milestones, attributes for metadata.

Notice that `validateOrder()` doesn't get a span – it's pure computation. The validation timing is captured as an event attribute if you need it. The database insert and payment call **do** get spans because they're I/O operations where latency matters.



✓ SUCCESS

Rule of thumb: if it involves I/O (network, disk, database), make it a span. If it's a milestone within an operation, make it an event. If it's metadata about the operation, make it an attribute. Spans are expensive; events and attributes are cheap.

Performance Overhead

Measuring Span Costs

Span overhead is real but often misunderstood. People either ignore it entirely ("tracing is free") or overestimate it ("we can't afford instrumentation"). The actual cost depends on span count, attribute count, and throughput.

Here's what you're paying for with each span:

Span creation:

Costs 1-5 microseconds. You're allocating a span object, generating an 8-byte span ID, copying the parent context, and recording a start timestamp.

Attribute setting:

Costs 0.1-1 microsecond per attribute. Type conversion, string allocation, map insertion. Attribute count matters more than attribute size – ten small attributes cost more than one large one.

Span ending:

Costs 1-2 microseconds. Recording the end timestamp, calculating duration, enqueueing for export.

Export:

Amortized across batches (typically 512-2048 spans). Serialization to OTLP protobuf, network I/O to your collector, ingestion at your backend.

Memory:

Runs 200-500 bytes per in-flight span, depending on attribute count.

In aggregate, a single span with a few attributes costs roughly 5-10 microseconds of CPU time. That sounds negligible, but it multiplies:



Throughput	Spans/Request	Spans/Second	CPU Overhead	Action
100 RPS	20	2,000	<1%	None needed
1,000 RPS	20	20,000	1-3%	Monitor
10,000 RPS	20	200,000	5-15%	Sample
1,000 RPS	200	200,000	10-20%	Reduce spans

Overhead scales with span volume, not request volume.

The last row is the trap. Over-instrumented services (200 spans per request) hit the same overhead as high-throughput services (10,000 RPS) at a fraction of the traffic. The fix isn't sampling – it's reducing span count.

Sampling Strategies

When overhead becomes a concern, sampling lets you capture representative traces without instrumenting everything. There are two fundamental approaches: head sampling (decide at span creation) and tail sampling (decide after seeing the complete trace).

1

Head sampling

Decides whether to create spans before the request starts. The upside is zero overhead for dropped traces – if you're not sampling this request, you don't create any spans. The downside is you might drop interesting traces (errors, slow requests) because you couldn't know they'd be interesting when sampling occurred.

The most common head sampler is ratio-based: sample 10% of traces by hashing the trace ID. Parent-based sampling builds on this – if an upstream service already sampled a trace, continue sampling; if they dropped it, you drop it too. This keeps distributed traces coherent.



head-sampling.ts

```

1  // Head sampling configurations
2  import { TraceIdRatioBasedSampler, ParentBasedSampler } from '@opentelemetry/sdk-trace-base';
3
4  // Sample 10% of traces by trace ID hash
5  const ratioSampler = new TraceIdRatioBasedSampler(0.1);
6
7  // Respect parent's decision, use 10% ratio for root spans
8  const parentBasedSampler = new ParentBasedSampler({
9    root: new TraceIdRatioBasedSampler(0.1),
10  });

```

Head sampling with ratio and parent-based samplers.

2

Tail sampling

Decides after seeing the complete trace. This happens in your collector, not your application. The collector buffers traces for a few seconds, then applies policies: keep all errors, keep slow traces, keep traces from enterprise customers, drop the rest. The upside is you never miss interesting traces. The downside is you still pay the overhead of creating all those spans – you're just dropping them before storage.

otel-collector-tail-sampling.yaml

```

1  # OpenTelemetry Collector tail sampling configuration
2  processors:
3    tail_sampling:
4      decision_wait: 10s
5      num_traces: 100000
6      policies:
7        - name: errors
8          type: status_code
9          status_code: { status_codes: [ERROR] }
10       - name: slow-traces
11         type: latency
12         latency: { threshold_ms: 1000 }
13       - name: high-value
14         type: string_attribute

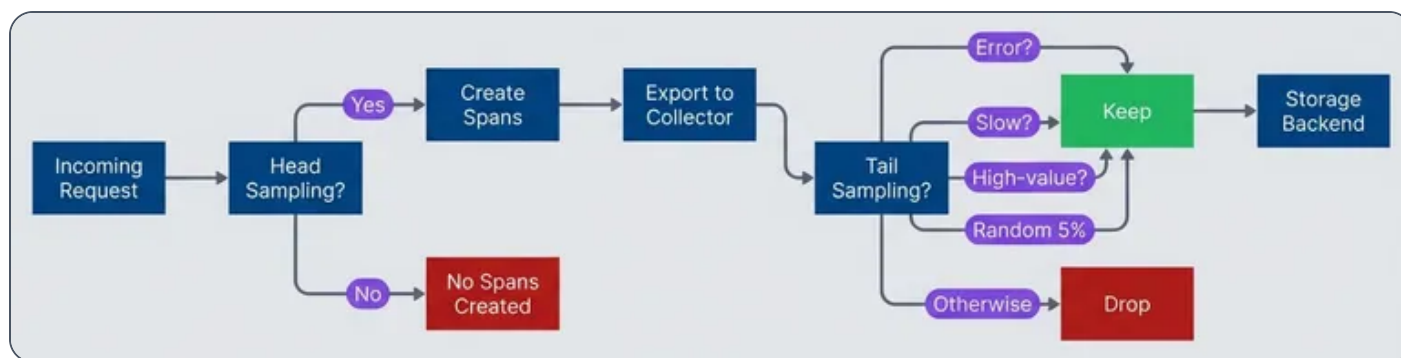
```



```
15     string_attribute: { key: customer.tier, values: [enterprise] }
16   - name: baseline
17     type: probabilistic
18     probabilistic: { sampling_percentage: 5 }
```

Tail sampling in the OpenTelemetry Collector.

The typical pattern: use head sampling for high-throughput services where overhead matters, use tail sampling when you can afford the overhead but want comprehensive error capture. The following diagram shows how these two approaches interact in a typical pipeline:



Head sampling reduces overhead - tail sampling preserves interesting traces.

⚠ WARNING

Head sampling reduces instrumentation overhead but might drop interesting traces. Tail sampling sees complete traces but requires buffering and doesn't reduce application overhead. Use head sampling for high-throughput services, tail sampling for comprehensive error capture.

Trace Readability

Waterfall Anti-Patterns

A trace is only useful if you can read it. I've seen traces that technically contain all the information needed to debug a problem, but the information is buried in noise. Here are the anti-patterns to avoid.



➤ **Wall of spans.**

The waterfall is solid color – no gaps, no white space. You can't see the structure because there's a span for everything. Every function call, every loop iteration, every trivial operation. The cause is over-instrumentation. The fix is reducing span count and using events for milestones instead of child spans.

➤ **Flat hierarchy.**

All spans at the same level, no nesting. You can't tell which operations are contained within others, or whether operations ran sequentially or in parallel. The cause is usually not using `startActiveSpan` (which automatically sets parent context) or not propagating context to background tasks. The fix is ensuring child spans inherit from their parents.`

➤ **Missing gaps.**

Spans account for 100% of request time – no uninstrumented periods visible. This sounds good but actually hides information. The gaps in a trace show where time went to uninstrumented code (JSON parsing, business logic, framework overhead). If there are no gaps, you can't distinguish "the database was slow" from "the instrumentation overhead was high."

➤ **Cryptic names.**

Span names like "span," "operation," "handler," or "process" that don't explain what's happening. Auto-instrumentation often produces these – you get a span for every HTTP call, but the name is just "GET" without any indication of what resource. The fix is customizing span names to follow the pattern "operation resource" (HTTP GET /api/orders, db.query orders).

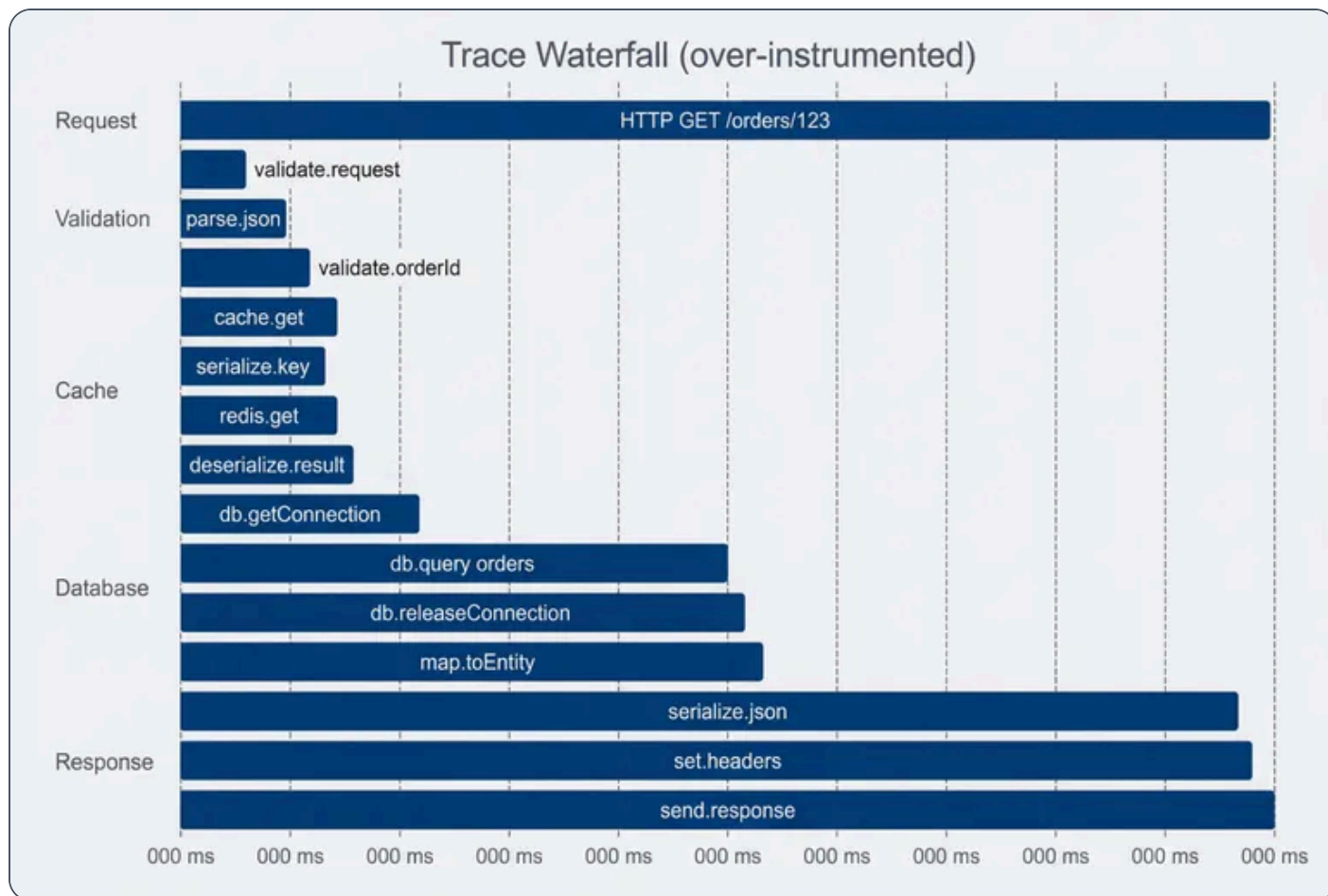
➤ **Attribute explosion.**

Hundreds of attributes per span because someone dumped all available context. The trace viewer loads slowly, important attributes are buried in noise, and storage costs balloon. The fix is selecting relevant attributes deliberately, not including everything available.

A good trace has 10-30 spans per request, 3-5 levels of nesting, clear names that explain each operation, and visible gaps that show where uninstrumented time went. You should be able to identify the critical path (the longest sequence of spans) at a glance.

Compare this over-instrumented trace to the readable one shown earlier:





Over-instrumented trace - wall of spans obscures the critical path.

The over-instrumented trace has 15 spans where 4 would suffice. You can't see at a glance that the database query (`db.query orders`) took 57ms and dominated the request. The signal is buried in noise.

Naming Conventions

Span names should answer “what operation on what resource?” without requiring you to read the code. The OpenTelemetry semantic conventions provide a good starting point.

For HTTP servers:

Use HTTP `{method} {route}`: HTTP GET `/api/orders/{orderId}`. The route should be parameterized (with `{orderId}`, not the actual ID) to keep cardinality low.



For HTTP clients:

Use HTTP `{method}` with the target service in an attribute: HTTP GET with `peer.service: payments-api`.



For databases:

Use `db.{operation} {table}`: `db.query orders`, `db.insert users`. Don't include the full SQL query in the name – that goes in an attribute if needed.



For caches:

Use `cache.{operation}`: `cache.get`, `cache.set`. Include the key pattern in an attribute.



For internal operations:

Use `{domain}.{operation}`: `order.validate`, `payment.process`, `inventory.reserve`.



Context	Good Name	Bad Name	Problem with Bad Name
HTTP handler	HTTP GET <code>/api/orders/{id}</code>	<code>handler</code>	No indication of method or resource
Database	<code>db.query orders</code>	<code>SELECT *</code> <code>FROM...</code>	SQL syntax noise, potential sensitive data
Cache	<code>cache.get</code>	<code>redis</code>	Technology not operation – what did you do?
Business logic	<code>order.validate</code>	<code>validateOrder</code>	Function name leaks implementation detail
External API	HTTP POST	<code>API call</code>	No method, no way to distinguish calls

Span naming patterns.



 INFO

Span names should answer “what operation on what resource?” at a glance. A developer should understand the trace without reading code. Use the OpenTelemetry semantic conventions as a starting point.

Attribute Design

Selecting Attributes

Attributes carry the metadata that makes traces useful. Choose them carefully – every attribute consumes storage, affects query performance, and either helps or clutters your debugging experience.

➤ **Essential attributes:**

Belong on every relevant span. Identity attributes (`service.name`, `service.version`) let you filter traces by service and compare behavior across deployments – trace and span IDs are added automatically. Request attributes (`http.method`, `http.route`, `http.status_code`) describe the operation. User attributes (`user.id`, `user.role`, `tenant.id`) enable filtering by customer segment. Business attributes (`order.id`, `customer.tier`, `feature.flag`) connect technical traces to domain concepts.

➤ **Contextual attributes:**

Add value in specific situations. Performance attributes (`db.rows_affected`, `cache.hit`, `retry.count`, `queue.depth`) help explain why an operation was slow. Debugging attributes (`error.type`, `error.message`, `exception.stacktrace`) accelerate root cause analysis. Business attributes like `payment.amount` or `order.item_count` provide context when investigating specific transactions.

➤ **Avoid certain attribute categories entirely:**

PII (email addresses, names, credit card numbers, full IP addresses) creates compliance risk and rarely helps debugging – use pseudonymous identifiers instead. High-cardinality values (request IDs, timestamps, session IDs) bloat indexes without adding queryable value – use the trace ID for correlation, not a separate request ID. Large values (full request bodies, complete SQL queries) waste storage and obscure other attributes – log these elsewhere if needed. Redundant data (duration, parent span ID) duplicates information already in the span structure.



➤ **Cardinality determines query performance:**

Low-cardinality attributes (under 100 unique values) like `http.method` or `http.status_code` index efficiently and query quickly. Medium-cardinality attributes (100-10,000 values) like `http.route` or `tenant.id` work acceptably for most backends. High-cardinality attributes (over 10,000 values) like timestamps or UUIDs cause index bloat and slow queries — avoid them unless essential.

Attribute Value Patterns

How you format attribute values matters as much as which attributes you choose. These patterns keep cardinality low while preserving useful information:

attribute-patterns.ts

```
1  import { trace, SpanStatusCode, Span } from '@opentelemetry/api';
2
3  // Pattern 1: Normalize high-cardinality values
4  function setHttpRouteAttribute(span: Span, path: string) {
5    // Bad: /api/users/123 (high cardinality)
6    // Good: /api/users/{userId} (low cardinality)
7    const normalizedRoute = path
8      .replace(/\/users\/[a-f0-9-]+/g, '/users/{userId}')
9      .replace(/\/orders\/[a-f0-9-]+/g, '/orders/{orderId}')
10     .replace(/\/\d+/g, '/{id}');
11
12    span.setAttribute('http.route', normalizedRoute);
13    span.setAttribute('resource.id', extractId(path)); // Keep actual ID separately
14  }
15
16  // Pattern 2: Bucket continuous values
17  function setLatencyBucket(span: Span, durationMs: number) {
18    // Bad: 147.23 (infinite cardinality)
19    // Good: "100-200ms" (finite buckets)
20    const buckets = [0, 10, 50, 100, 200, 500, 1000, 5000, Infinity];
21    const bucket = buckets.find((b, i) => durationMs < buckets[i + 1]);
22    span.setAttribute('latency.bucket', `${bucket}-${buckets[buckets.indexOf(bucket) + 1]}ms`);
23  }
24
25  // Pattern 3: Redact sensitive data
26  function setUserAttribute(span: Span, user: User) {
27    span.setAttribute('user.id', user.id); // Pseudonymous identifier
```



```
28     span.setAttribute('user.tier', user.tier);
29     // Don't include: email, name, phone, address
30 }
31
32 // Pattern 4: Structured error handling
33 function setErrorAttributes(span: Span, error: Error) {
34     span.setAttribute('error.type', error.constructor.name);
35     span.setAttribute('error.message', error.message.slice(0, 500)); // Truncate
36     span.recordException(error); // Full stack trace goes here, not as attribute
37     span.setStatus({ code: SpanStatusCode.ERROR, message: error.message });
38 }
39
40 // Pattern 5: Boolean and enum attributes
41 function setOutcomeAttributes(span: Span, result: Result) {
42     span.setAttribute('cache.hit', result.fromCache); // Boolean
43     span.setAttribute('result.status', result.status); // Enum: 'success',
44     'failure'
45     span.setAttribute('result.source', result.source); // Enum: 'cache',
46     'database'
47 }
```

Attribute value patterns.

⚠ WARNING

High-cardinality attributes kill trace backend performance. Every unique attribute value becomes an index entry. Use parameterized routes (`/users/{id}` not `/users/123`), bucket continuous values, and avoid timestamps or UUIDs as attribute values.

Practical Patterns

Request Lifecycle Instrumentation

A well-instrumented request follows a predictable pattern: extract incoming trace context, create a root span for your service, add child spans for I/O operations, propagate context on outgoing calls, and record the outcome.



1

Middleware creates the root span.

The tracing middleware extracts any incoming trace context (so your service continues an existing trace rather than starting a new one), creates the root span with `SpanKind.SERVER`, and ensures the span ends when the response finishes – regardless of success or failure.

tracing-middleware.ts

```
1  import { trace, context, SpanKind, SpanStatusCode, propagation } from
    '@opentelemetry/api';
2  import express from 'express';
3
4  const tracer = trace.getTracer('api-service', '1.0.0');
5
6  function tracingMiddleware(req: express.Request, res: express.Response, next:
    express.NextFunction) {
7      const extractedContext = propagation.extract(context.active(), req.headers);
8
9      const span = tracer.startSpan(
10         `HTTP ${req.method} ${normalizeRoute(req.path)}`,
11         {
12             kind: SpanKind.SERVER,
13             attributes: {
14                 'http.method': req.method,
15                 'http.route': normalizeRoute(req.path),
16                 'service.name': 'api-service',
17             },
18         },
19         extractedContext
20     );
21
22     req.span = span;
23
24     res.on('finish', () => {
25         span.setAttribute('http.status_code', res.statusCode);
26         span.setStatus(res.statusCode >= 400
27             ? { code: SpanStatusCode.ERROR, message: `HTTP ${res.statusCode}` }
28             : { code: SpanStatusCode.OK });
29         span.end();
30     });
31
32     context.with(trace.setSpan(context.active(), span), () => next());
```



33 }

Tracing middleware creates the root span.

2

Route handlers add child spans for I/O.

With the middleware handling span lifecycle, route handlers focus on business logic. Create child spans only for I/O operations – cache lookups, database queries, external calls – not for in-memory computation.

route-handler.ts

```
1  async function getOrderHandler(req: express.Request, res: express.Response) {
2    const orderId = req.params.orderId;
3    req.span?.setAttribute('order.id', orderId);
4
5    try {
6      // Child span: cache lookup
7      const cached = await tracer.startActiveSpan('cache.get', async (span) => {
8        const result = await cache.get(`order:${orderId}`);
9        span.setAttribute('cache.hit', result !== null);
10       span.end();
11       return result;
12     });
13
14     if (cached) return res.json(cached);
15
16     // Child span: database query
17     const order = await tracer.startActiveSpan('db.query orders', async (span) => {
18       span.setAttribute('db.system', 'postgresql');
19       const result = await db.query('SELECT * FROM orders WHERE id = $1', [orderId]);
20       span.setAttribute('db.rows_affected', result.rowCount);
21       span.end();
22       return result.rows[0];
23     });
24
25     if (!order) {
26       req.span?.setStatus({ code: SpanStatusCode.ERROR, message: 'Not found' });
27       return res.status(404).json({ error: 'Order not found' });
28     }
29
30     res.json(order);
```




```
31     } catch (error) {
32       req.span?.recordException(error);
33       req.span?.setStatus({ code: SpanStatusCode.ERROR, message: error.message });
34       res.status(500).json({ error: 'Internal server error' });
35     }
36   }
```

Route handler with child spans for I/O operations.

3

External calls propagate context.

When calling other services, inject the current trace context into outgoing headers. This connects spans across service boundaries into a single distributed trace.

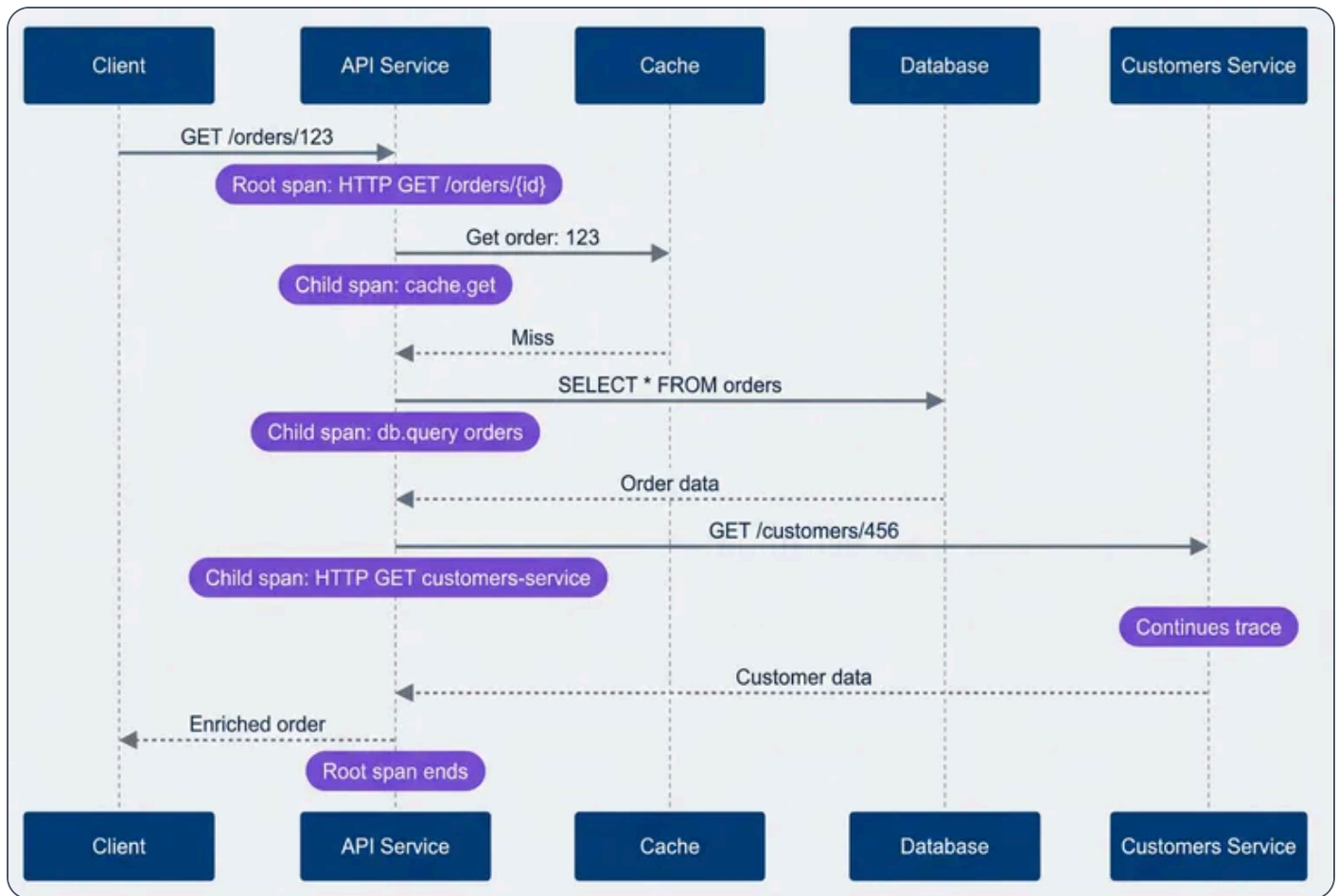
external-call.ts

```
1  async function fetchCustomer(customerId: string): Promise<Customer> {
2    return tracer.startActiveSpan('HTTP GET customers-service', { kind: SpanKind.CLIENT },
3      async (span) => {
4        span.setAttribute('peer.service', 'customers-service');
5
6        const headers: Record<string, string> = {};
7        propagation.inject(context.active(), headers);
8
9        try {
10           const response = await fetch(`http://customers-service/customers/${customerId}`, {
11             headers });
12           span.setAttribute('http.status_code', response.status);
13           return response.ok ? response.json() : null;
14         } finally {
15           span.end();
16         }
17       }
18   );
19 }
```

Context propagation on external calls.

The sequence diagram shows how these pieces fit together. The root span encompasses the entire request, child spans capture individual I/O operations, and context propagation links the trace across service boundaries.





Request instrumentation sequence.

Batch Processing Instrumentation

Batch processing is where span proliferation gets dangerous. Creating a span per item in a 10,000-item batch generates 10,000 spans – overwhelming your collector, bloating storage, and producing unreadable waterfalls.

The correct pattern: one span for the batch.

Wrap the entire batch in a single span. Record individual failures as events rather than child spans.
Use attributes to capture summary statistics.



batch-single-span.ts

```
1  import { trace, SpanKind, SpanStatusCode } from '@opentelemetry/api';
2
3  const tracer = trace.getTracer('batch-processor');
4
5  async function processBatch(items: Item[]): Promise<BatchResult> {
6    return tracer.startActiveSpan('batch.process', { kind: SpanKind.CONSUMER }, async
    (span) => {
7      span.setAttribute('batch.size', items.length);
8
9      const results = { successful: 0, failed: 0 };
10
11     for (const item of items) {
12       try {
13         await processItem(item);
14         results.successful++;
15       } catch (error) {
16         results.failed++;
17         // Record failure as event, not child span
18         span.addEvent('item.failed', { 'item.id': item.id, 'error.message':
            error.message });
19       }
20     }
21
22     span.setAttribute('batch.successful', results.successful);
23     span.setAttribute('batch.failed', results.failed);
24     span.setStatus(results.failed > 0
25       ? { code: SpanStatusCode.ERROR, message: `${results.failed} items failed` }
26       : { code: SpanStatusCode.OK });
27     span.end();
28
29     return results;
30   });
31 }
```

Single span for a batch with events for failures.



For very large batches: spans per chunk.

If you need more granularity than a single batch span provides, create spans for chunks of 10-100 items – never for individual items. This gives you visibility into which portion of the batch had problems without creating thousands of spans.



batch-chunked.ts

```
1  async function processLargeBatch(items: Item[]): Promise<BatchResult> {
2    const chunks = chunkArray(items, 100);
3
4    return tracer.startActiveSpan('batch.process', async (batchSpan) => {
5      batchSpan.setAttribute('batch.total_items', items.length);
6      batchSpan.setAttribute('batch.chunk_count', chunks.length);
7
8      let totalSuccessful = 0, totalFailed = 0;
9
10     for (let i = 0; i < chunks.length; i++) {
11       await tracer.startActiveSpan('batch.chunk', async (chunkSpan) => {
12         chunkSpan.setAttribute('chunk.index', i);
13         const results = await processChunk(chunks[i]);
14         chunkSpan.setAttribute('chunk.successful', results.successful);
15         chunkSpan.end();
16
17         totalSuccessful += results.successful;
18         totalFailed += results.failed;
19       });
20     }
21
22     batchSpan.setAttribute('batch.successful', totalSuccessful);
23     batchSpan.setAttribute('batch.failed', totalFailed);
24     batchSpan.end();
25     return { successful: totalSuccessful, failed: totalFailed };
26   });
27 }
```

Chunked spans for large batches.



The anti-pattern: span per item.

This looks intuitive but scales terribly. A 10,000-item batch creates 10,000 spans, each with overhead for creation, context propagation, and export.



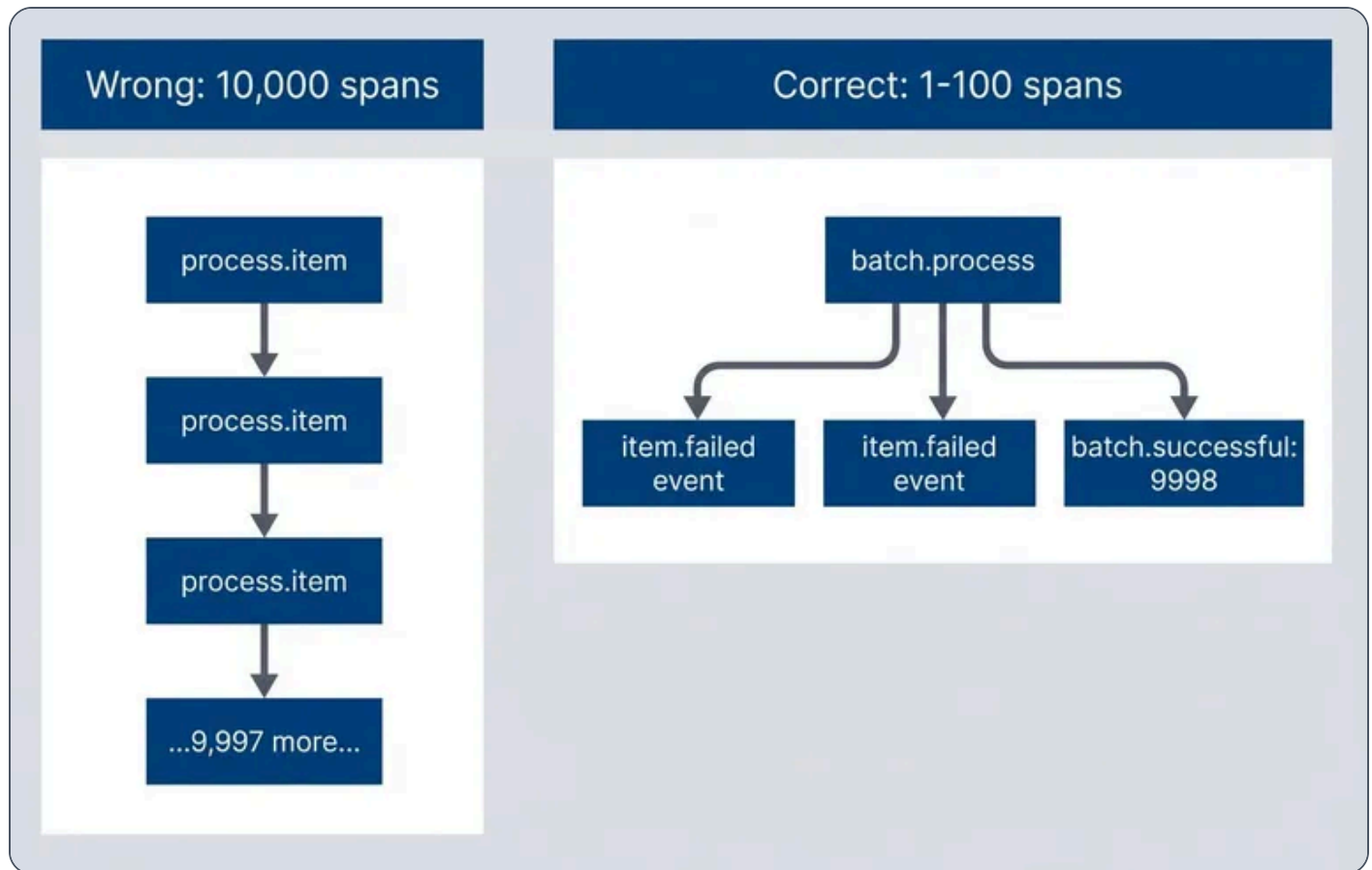
batch-wrong.ts

```
1  // WRONG: Creates thousands of spans
2  async function processBatchWrong(items: Item[]): Promise<void> {
3    for (const item of items) {
4      await tracer.startActiveSpan('process.item', async (span) => {
5        await processItem(item);
6        span.end();
7      });
8    }
9  }
```

Anti-pattern – span per item.

The difference in trace output is stark:





Batch instrumentation approaches – events vs spans.

✓ SUCCESS

For batch processing: one span for the batch, events for individual failures, attributes for summary statistics. If you need finer granularity, use spans for chunks (10-100 items), not individual items. A 10,000-item batch should create 1-100 spans, not 10,000.

Conclusion

Span design is an engineering tradeoff: visibility versus overhead, granularity versus readability, detail versus cost. The goal isn't maximum spans – it's enough spans to debug problems efficiently.



Instrument service boundaries, I/O operations, and significant business logic. Use events for milestones within spans. Use attributes for metadata that helps filtering and debugging. Sample aggressively in high-throughput services – you don’t need every trace, just enough to catch problems. Follow naming conventions so waterfalls tell a story at a glance.

INFO

Start with minimal instrumentation – auto-instrumentation plus key business operations – then add spans only when you can’t debug a specific problem. You can always add granularity; removing it requires code changes. Let debugging needs drive instrumentation, not the quest for “complete” visibility.

The best traces answer three questions: what happened, where time was spent, and what failed. The worst traces are walls of noise that hide the signal you need.

Start by auditing your current traces: pick a typical request, count the spans, and ask whether you could identify the slow operation in under 10 seconds. If not, you’ve found your first refactoring target.

Copyright © 2026 Webstack Builders, Inc.

The text, diagrams, and images in this work are licensed under CC BY-NC 4.0

All code samples in this article are licensed under the MIT License. Feel free to use, modify, and distribute them in any project.

<https://www.webstackbuilders.com/articles/opentelemetry-span-design-granularity-overhead>

